

lecture_13

August 31, 2026

1 Lecture 13: Unconstrained Optimization — Gradient Descent

{note} Lecture 12 introduced the NLP landscape and established that finding a minimum of a smooth function requires the gradient to vanish - $\nabla f(x^*) = \mathbf{0}$ - with the Hessian confirming the character of the stationary point. But that tells us *where* the optimum is, not *how to find it*. This lecture introduces Gradient Descent, the simplest iterative algorithm for unconstrained minimization.

Learning Objectives

By the end of this notebook, you will be able to: 1. Apply first- and second-order optimality conditions to classify stationary points of a smooth unconstrained objective function as local minima, local maxima, or saddle points. 2. Trace the Gradient Descent algorithm step-by-step and explain the role of the step size in convergence. 3. Explain the trade-offs in convergence speed and computational cost across Gradient Descent, Newton's Method, and BFGS.

Prerequisites: NLP Principles (Lecture 12); calculus (partial derivatives, Taylor expansion); basic linear algebra (matrix inverse, eigenvalues).

Estimated time: 90 minutes (including in-class exercises)

1.1 Optimality Conditions

1.1.1 First-Order Necessary Condition

If x^* is a local optimum of a differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, then the gradient:

$$\nabla f(x^*) = \mathbf{0}$$

A point satisfying $\nabla f(x^*) = \mathbf{0}$ is called a stationary point. This is a necessary condition, not sufficient — a stationary point can be a minimum, a maximum, or a saddle point.

1.1.2 Second-Order Conditions

The Hessian $\nabla^2 f(x^*)$ — the $n \times n$ matrix of second partial derivatives — tells us the curvature of f at the stationary point and resolves the ambiguity:

Hessian $\nabla^2 f(x^*)$	All eigenvalues	Character of x^*
Positive definite (PD)	> 0	Local minimum
Negative definite (ND)	< 0	Local maximum
Indefinite	Mixed signs	Saddle point

1.1.3 Example

A logistics firm wants to locate a cross-dock hub at (x, y) to minimize total weighted distance to three demand zones in the Chennai region. Using squared Euclidean distance as a differentiable proxy:

Demand zone	Location (a_k, b_k) km (relative to Tambaram)	Weight w_k (tonnes/day)
Sriperumbudur	$(-30, 10)$	40
Maraimalai Nagar	$(-10, -15)$	25
Chengalpattu	$(5, -40)$	35

$$f(x, y) = \sum_{k=1}^3 w_k [(x - a_k)^2 + (y - b_k)^2]$$

Setting $\nabla f = \mathbf{0}$:

$$x^* = \frac{\sum_k w_k a_k}{\sum_k w_k} = \frac{40(-30) + 25(-10) + 35(5)}{100} = -12.75 \text{ km}$$

$$y^* = \frac{\sum_k w_k b_k}{\sum_k w_k} = \frac{40(10) + 25(-15) + 35(-40)}{100} = -13.75 \text{ km}$$

The Hessian is $\nabla^2 f = 2(\sum_k w_k) \mathbf{I} = 200 \mathbf{I}$, which is positive definite. Hence $(x^*, y^*) = (-12.75, -13.75)$ is a global minimum — the optimal hub lies approximately 12.75 km west and 13.75 km south of Tambaram.

1.2 Iterative Descent Framework

For a general nonlinear function f , the system of equations $\nabla f(x) = \mathbf{0}$ may be nonlinear with no closed-form solution. To this end, to find the point where the gradient vanishes for such problems, we start from an initial guess $x^{(0)}$ and repeatedly compute a direction $d^{(k)}$ that moves downhill, then take a step along it $x^{(k+1)} = x^{(k)} + \alpha^{(k)} d^{(k)}$, until the gradient is (approximately) zero. The three methods covered in Lectures 13–15 differ only in how they choose $d^{(k)}$:

Method	Direction $d^{(k)}$	Information used	Convergence
Gradient Descent	$-\nabla f(x^{(k)})$	First-order	Linear
Newton's Method	$-\nabla^2 f(x^{(k)})^{-1} \nabla f(x^{(k)})$	Second-order	Quadratic
Quasi-Newton's Method (BFGS)	$-\nabla f(x^{(k)}), H^{(k)} \approx \nabla^2 f$	Accumulated first-order	Superlinear

`{tip}` In practice: solve $\nabla f(x) = \mathbf{0}$ analytically if a closed-form solution can be established; otherwise, default to Quasi-Newton's Method; use Newton's Method when the Hessian is cheap to compute and invert; use Gradient Descent when problem scale makes storing a Hessian prohibitive.

1.3 1. Gradient Descent

Gradient Descent is the simplest iterative descent method. It moves in the direction of steepest descent at the current point — the negative gradient — and requires only first-order information:

$$d^{(k)} = -\nabla f(x^{(k)})$$

$$x^{(k+1)} = x^{(k)} - \alpha^{(k)} \nabla f(x^{(k)})$$

The negative gradient points in the direction in which f decreases fastest locally — making it a natural, if myopic, choice. The gradient carries no information about curvature, so the method must compensate through careful selection of the **step size** $\alpha^{(k)}$:

- Static: fixed across all iterations; simple, but risks overshooting (too large) or crawling (too small).
- Dynamic: at each iteration, find $\alpha^{(k)} = \arg \min_{\alpha > 0} f(x^{(k)} - \alpha \nabla f(x^{(k)}))$; more robust and used in practice.

Gradient Descent converges linearly: the error reduces by a constant factor per iteration. Near the optimum, where gradients are small and the landscape may be ill-conditioned, convergence can slow dramatically — a consequence of having no curvature information.

1.4 In-Class Exercises

1.4.1 Exercise 1

An NHAH corridor study models total system travel time across two parallel links — the Mumbai–Pune Expressway (v_1) and old NH48 (v_2), with volumes in thousands of vehicles/hour — as:

$$T(v_1, v_2) = 3v_1^2 - 2v_1v_2 + 2v_2^2 - 14v_1 - 12v_2 + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 3 iterations of Gradient Descent by hand, starting from $(v_1, v_2) = (0, 0)$ with step size $\alpha = 0.1$.

1.4.2 Exercise 2

Delhivery is minimizing operational cost at its Chennai cross-dock facility. The daily cost as a function of throughput λ (tonnes/day) and staffing level s (workers) is:

$$C(\lambda, s) = 2\lambda^2 - 4\lambda s + 3s^2 - 12\lambda - 6s + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
 2. Trace 3 iterations of Gradient Descent by hand, starting from $(\lambda, s) = (0, 0)$ with step size $\alpha = 0.1$.
-

1.5 Take-Away Exercises

1.5.1 Exercise 1

BMTC is optimizing its demand-responsive feeder service in Bengaluru. The total daily operating cost depends on fleet size n (vehicles) and headway h (minutes). The cost model is:

$$C(n, h) = 2n^2 + 3h^2 + 2nh - 20n - 30h + 100$$

where the terms capture vehicle capital cost, passenger waiting cost, and a joint fleet-frequency interaction respectively.

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 5 iterations of Gradient Descent by hand, starting from $(n, h) = (0, 0)$ with step size $\alpha = 0.1$.
3. At the optimum, BMTC's analyst claims: "Deploying one additional vehicle always reduces total cost." Verify by evaluating $\partial C / \partial n$ at (n^*, h^*) .

1.5.2 Exercise 2

An NHAI planning study models the full-lifecycle cost of a four-lane corridor as a function of traffic volume v (thousand veh/day) and maintenance frequency m (interventions/year):

$$F(v, m) = 3v^2 - 2vm + 2m^2 - 18v - 12m + 80$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Implement Gradient Descent in Python using `scipy.optimize.minimize` with `method='CG'`, starting from $(v, m) = (0, 0)$. Report the optimal solution and number of iterations.
3. NHAI proposes a volume cap of $v \leq 4$ thousand veh/day. At the unconstrained optimum, is this constraint binding? What does this imply for whether a constrained solve is necessary?

1.6 Further Reading

- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 2 (fundamentals of unconstrained optimization), Chapter 3 (line search methods).
- Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. (2006). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). Wiley — Chapter 8 (gradient methods).
- SciPy documentation: `scipy.optimize.minimize` — docs.scipy.org
- BPR (1964). *Traffic Assignment Manual*. US Bureau of Public Roads, Washington DC.